Practical 1:

Import the NumPy library as np. Create a 1D NumPy array containing the numbers 10, 20, 30, 40, 50. Print the array and its data type (dtype).

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
print(arr)
print(arr.dtype)
```

```
[10 20 30 40 50]
int64
```

Practical 2:

Create a NumPy array of 6 integers. Print its shape (.shape), number of dimensions (.ndim), and total number of elements (.size).

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60])
print(arr.shape)
print(arr.ndim)
print(arr.size)
```

```
(6,)
1
6
```

Practical 3:

Use np.zeros() to create an array of 5 zeros, and np.ones() to create an array of 4 ones. Print both arrays.

```
import numpy as np

zeros = np.zeros(5)
ones = np.ones(4)

print(zeros)
print(ones)
```

```
[0. 0. 0. 0. 0.]
[1. 1. 1. 1.]
```

Practical 4:

Use np.arange() to generate a NumPy array containing all even numbers from 2 to 20 (inclusive). Print the array.

```
import numpy as np

arr = np.arange(2, 21, 2)
print(arr)
```

```
[ 2  4  6  8 10 12 14 16 18 20]
```

Practical 5:

Given the NumPy array arr = np.array([12, 25, 37, 48, 59, 61]), use positive indexing to print the 3rd element and negative indexing to print the last element.

```
import numpy as np

arr = np.array([12, 25, 37, 48, 59, 61])
print(arr[2])
print(arr[-1])
```

```
37
61
```

Practical 6:

Given arr = np.array([10, 20, 30, 40, 50, 60]), use array slicing arr[start:stop] to extract and print the sub-array [20, 30, 40].

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60])
print(arr[1:4])
```

```
[20 30 40]
```

Practical 7:

Create a NumPy array scores = np.array([45, 88, 92, 67, 73]). Calculate and print the average (mean), highest score, and lowest score using np.mean(), np.max(), and np.min().

```
import numpy as np

scores = np.array([45, 88, 92, 67, 73])

print(np.mean(scores))
print(np.max(scores))
print(np.min(scores))
```

```
73.0
92
45
```

Practical 8:

Given two NumPy arrays a = np.array([1, 2, 3]) and b = np.array([4, 5, 6]), perform element-wise addition, subtraction, and multiplication. Print all three results.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)
print(a - b)
print(a * b)
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
```

Practical 9:

Given a 1D NumPy array arr = np.array([1, 2, 3, 4, 5, 6]), use .reshape() to convert it into a 2D array of shape (2, 3) (2 rows, 3 columns). Print the reshaped array.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])
arr = arr.reshape(2, 3)

print(arr)
```

```
[[1 2 3]
 [4 5 6]]
```

Practical 10:

Create a dictionary inventory = {"laptop": 5, "mouse": 12, "keyboard": 8}. Write a function check_stock(item_name) that uses the in operator to check if the item exists and prints its quantity or "Out of Stock".

```
inventory = {"laptop": 5, "mouse": 12, "keyboard": 8}

def check_stock(item_name):
    if item_name in inventory:
        print(inventory[item_name])
    else:
        print("Out of Stock")

check_stock("laptop")
```

```
5
```

Practical 11:

Given a NumPy array prices = np.array([100, 250, 400, 150]), multiply the entire array by 1.18 to apply a 18% GST tax rate using vectorization. Print the tax-inclusive prices.

```
import numpy as np

prices = np.array([100, 250, 400, 150])
tax_prices = prices * 1.18

print(tax_prices)
```

```
[118. 295. 472. 177.]
```

Practical 12:

Given arr = np.array([15, 8, 22, 5, 30, 11]), write a program using boolean masking (arr > 10) to extract and print all numbers greater than 10.

```
import numpy as np

arr = np.array([15, 8, 22, 5, 30, 11])
print(arr[arr > 10])
```

```
[15 22 30 11]
```

Practical 13:

Write a function sum_array(arr) that accepts a NumPy array and calculates the sum of all its elements using a for loop. Compare your output with np.sum().

```
import numpy as np

def sum_array(arr):
    total = 0
    for i in arr:
        total += i
    return total

arr = np.array([10, 20, 30, 40])

print(sum_array(arr))
print(np.sum(arr))
```

```
100
100
```

Practical 14:

Given dict1 = {"A": 10, "B": 20} and dict2 = {"B": 30, "C": 40}, write a program to merge them into a new dictionary. If a key exists in both, sum their values.

```
dict1 = {"A": 10, "B": 20}
dict2 = {"B": 30, "C": 40}

result = dict1.copy()

for key, value in dict2.items():
    if key in result:
        result[key] += value
    else:
        result[key] = value

print(result)
```

```
{'A': 10, 'B': 50, 'C': 40}
```

Practical 15:

Create a tuple data = (5, 10, 15, 20). Convert this tuple into a NumPy array, multiply all elements by 2, and convert it back into a tuple. Print the final tuple.

```
import numpy as np

data = (5, 10, 15, 20)

arr = np.array(data)
arr = arr * 2
data = tuple(arr)

print(data)
```

```
(np.int64(10), np.int64(20), np.int64(30), np.int64(40))
```

Practical 16:

Write a program that uses np.arange(1, 10) to create a 1D array of digits, reshapes it into a 3x3 matrix, and prints the element located at row index 1, column index 2.

```
import numpy as np

arr = np.arange(1, 10)
matrix = arr.reshape(3, 3)

print(matrix)
print(matrix[1, 2])
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
6
```

Practical 17:

Given a NumPy array temps = np.array([28.5, 31.0, 29.2, 33.4, 27.8]), write a program to count how many days recorded temperatures strictly above 30.0 degrees using np.sum(temps > 30.0).

```
import numpy as np

temps = np.array([28.5, 31.0, 29.2, 33.4, 27.8])

count = np.sum(temps > 30.0)

print(count)
```

```
2
```

Practical 18:

Given records = {"E101": "Active", "E102": "Inactive", "E103": "Active"}, use .pop() in a while loop to process and remove inactive employees until none remain. Print the updated dictionary.

```
records = {"E101": "Active", "E102": "Inactive", "E103": "Active"}

while "Inactive" in records.values():
    for key in list(records):
        if records[key] == "Inactive":
            records.pop(key)

print(records)
```

```
{'E101': 'Active', 'E103': 'Active'}
```

Practical 19:

Create a 2D NumPy array of size 3x3 with numbers from 1 to 9. Write code to extract and print only the 2nd row and only the 3rd column.

```
import numpy as np

arr = np.arange(1, 10).reshape(3, 3)

print(arr[1])
print(arr[:, 2])
```

```
[4 5 6]
[3 6 9]
```

Practical 20:

Write a function replace_negatives(arr) that takes a NumPy array containing positive and negative numbers and replaces all negative values with 0 using boolean assignment (arr[arr < 0] = 0).

```
import numpy as np

def replace_negatives(arr):
    arr[arr < 0] = 0
    return arr

arr = np.array([10, -5, 20, -8, 30])

print(replace_negatives(arr))
```

```
[10  0 20  0 30]
```

Practical 21:

The Factory Sensor Calibration Analysis

A manufacturing unit tracks daily automated output logs across two assembly shifts stored as NumPy arrays:

shift_1 = np.array([120, 135, 150, 142, 110]) shift_2 =

np.array([115, 140, 148, 139, 125]).

- Calculate the total daily output by adding both shift arrays together.
- Compute the percentage contribution of shift_1 to the total output for each day usingelement-wise division and multiplication.
- Identify which days (index positions) had a combined output exceeding 280 unitsusing boolean indexing.

```
import numpy as np

shift_1 = np.array([120, 135, 150, 142, 110])
shift_2 = np.array([115, 140, 148, 139, 125])

total = shift_1 + shift_2
percentage = (shift_1 / total) * 100
days = np.where(total > 280)[0]

print("Total daily output:", total)
print("Shift 1 contribution:", percentage)
print("Days exceeding 280:", days)
```

```
Total daily output: [235 275 298 281 235]
Shift 1 contribution: [51.06382979 49.09090909 50.33557047 50.53380783 46.808510
Days exceeding 280: [2 3]
```

Practical 22:

The E-Commerce Price Tag Currency Converter

A global retail site stores product prices in USD using a dictionary: catalog = {"Headphones": 50, "Smartwatch": 150, "Speaker": 80, "Charger": 20}.

- Convert all prices into INR (assuming 1 USD = 83 INR) by converting the catalog valuesinto a NumPy array, performing scalar multiplication, and storing the results back into a new dictionary catalog_inr.
- Calculate the total value of all items in INR using np.sum().

- Print the updated catalog along with the total inventory value.

```
import numpy as np

catalog = {"Headphones": 50, "Smartwatch": 150, "Speaker": 80, "Charger": 20}

names = list(catalog.keys())
prices = np.array(list(catalog.values()))

inr_prices = prices * 83

catalog_inr = dict(zip(names, inr_prices))

print(catalog_inr)
print("Total inventory value:", np.sum(inr_prices))
```

```
{'Headphones': np.int64(4150), 'Smartwatch': np.int64(12450), 'Speaker': np.int6
Total inventory value: 24900
```

Practical 23:

The Smart Weather Station Anomaly Detector

A meteorology station logs hourly humidity percentages over a 24-hour cycle in a NumPy array:

humidity = np.array([45, 48, 50, 52, 55, 60, 62, 65, 70, 75, 82, 85, 88, 90, 84, 78, 72, 68, 64, 58, 54, 50, 48, 46]).

- Reshape the 1D array of 24 readings into a 2D array of shape (4, 6) representing 6-hour quarters of the day.
- Write a function analyze_quarter(matrix) that calculates and returns the averagehumidity for each of the 4 quarters.
- Print an alert if any quarter's average humidity exceeds 80%.

```
import numpy as np

humidity = np.array([45, 48, 50, 52, 55, 60, 62, 65, 70, 75, 82, 85, 88, 90, 84

matrix = humidity.reshape(4, 6)

def analyze_quarter(matrix):
    return np.mean(matrix, axis=1)

averages = analyze_quarter(matrix)
```

```
print(matrix)
print("Quarter averages:", averages)

if np.any(averages > 80):
    print("Alert: Average humidity exceeds 80%")
```

```
[[45 48 50 52 55 60]
 [62 65 70 75 82 85]
 [88 90 84 78 72 68]
 [64 58 54 50 48 46]]
Quarter averages: [51.66666667 73.16666667 80.         53.33333333]
```

Practical 24:

The Student Grade Normalization System

A professor records final exam marks out of 80 for a class in a NumPy array: raw_marks = np.array([55, 72, 38, 64, 78, 42, 50, 80]).

- Scale all marks to a 100-point scale by multiplying the array by (100 / 80).
- Define a function grade_bracket(scaled_marks) that loops through the array andcategorizes students:

○ Marks ≥ 75: "Distinction"

○ 50 ≤ Marks < 75: "Pass"

○ Marks < 50: "Remedial"

- Store these counts in a dictionary summary = {"Distinction": 0, "Pass": 0, "Remedial":0} and print the final distribution.

```
    import numpy as np

    raw_marks = np.array([55, 72, 38, 64, 78, 42, 50, 80])
    scaled_marks = raw_marks * (100 / 80)

    def grade_bracket(scaled_marks):
        summary = {"Distinction": 0, "Pass": 0, "Remedial": 0}

        for mark in scaled_marks:
            if mark >= 75:
                summary["Distinction"] += 1
            elif mark >= 50:
                summary["Pass"] += 1
            else:
                summary["Remedial"] += 1

        return summary
```

```
print(scaled_marks)
print(grade_bracket(scaled_marks))
```

```
[ 68.75  90.    47.5   80.    97.5   52.5   62.5  100.  ]
{'Distinction': 4, 'Pass': 3, 'Remedial': 1}
```

Practical 25:

The Retail Store Inventory Restock Flag A supermarket tracks item stock quantities and reorder thresholds in separate NumPy arrays:

current_stock = np.array([15, 3, 40, 8, 2, 25]) min_required = np.array([10, 5, 20, 10, 5, 15]).

- Item names are stored in a tuple: items = ("Milk", "Bread", "Rice", "Butter", "Eggs","Cereal").
- Use NumPy comparison operators to generate a boolean array indicating which itemsare below their minimum required threshold (current_stock < min_required).
- Loop using zip(items, reorder_mask) and print the names of all items that needimmediate restocking.

```
import numpy as np

current_stock = np.array([15, 3, 40, 8, 2, 25])
min_required = np.array([10, 5, 20, 10, 5, 15])

items = ("Milk", "Bread", "Rice", "Butter", "Eggs", "Cereal")

reorder_mask = current_stock < min_required

for item, need in zip(items, reorder_mask):
    if need:
        print(item)
```

```
Bread
Butter
Eggs
```

Practical 26:

The Fitness Tracker Performance Evaluator

A wearable band logs daily step counts over a 10-day period in a NumPy array:

step_logs = np.array([8500, 10200, 6400, 9800, 12100, 4500, 11000, 9300, 7800, 10500]).

- Define a target goal of 10,000 steps per day.

- Use a boolean mask to calculate how many total days the user met or exceeded theirdaily goal.
- Calculate the mean step count across active days (days where steps ≥ 10,000) versus inactive days (days where steps < 10,000).
- Print both averages alongside a fitness summary message.

```
import numpy as np

step_logs = np.array([8500, 10200, 6400, 9800, 12100, 4500, 11000, 9300, 7800,

target = 10000

active = step_logs >= target
inactive = step_logs < target

days_met = np.sum(active)
active_average = np.mean(step_logs[active])
inactive_average = np.mean(step_logs[inactive])

print("Days goal met:", days_met)
print("Active days average:", active_average)
print("Inactive days average:", inactive_average)
print("Fitness Summary: Goal met on", days_met, "days")
```

```
Days goal met: 4
Active days average: 10950.0
Inactive days average: 7716.666666666667
Fitness Summary: Goal met on 4 days
```

Practical 27:

The Cinema Hall Ticket Booking Grid

A movie theater models seat availability for a screen using a 2D NumPy array of shape (5, 5) initialized with zeros (0 = Available, 1 = Booked): seats = np.zeros((5, 5), dtype=int).

- Mark seats at (0, 2), (1, 3), (2, 2), and (4, 1) as booked (1).
- Write a function book_seat(grid, row, col) that takes row and column numbers from auser:

○ If the seat is available (0), set it to 1 and return "Seat booked successfully!".

○ If the seat is already 1, return "Seat occupied. Choose another."

- Print the updated seat grid matrix after processing the booking attempt.

```
import numpy as np

seats = np.zeros((5, 5), dtype=int)

seats[0, 2] = 1
seats[1, 3] = 1
seats[2, 2] = 1
seats[4, 1] = 1

def book_seat(grid, row, col):
    if grid[row, col] == 0:
        grid[row, col] = 1
        return "Seat booked successfully!"
    else:
        return "Seat occupied. Choose another."

row = int(input("Enter row: "))
col = int(input("Enter column: "))

print(book_seat(seats, row, col))
print(seats)
```

```
Enter row: 2
Enter column: 1
Seat booked successfully!
[[0 0 1 0 0]
 [0 0 0 1 0]
 [0 1 1 0 0]
 [0 0 0 0 0]
 [0 1 0 0 0]]
```

Practical 28:

The Digital Signal Processing Noise Filter

An audio processing module records a noisy digital audio signal as floating-point values: signal = np.array([0.12, 0.85, 2.45, -1.20, 0.95, 3.10, -0.05, 0.60]).

- High-voltage spikes above 2.0 or below -1.0 are considered signal noise artifacts.
- Write a function clean_signal(sig_arr, lower_bound, upper_bound) that uses booleanindexing to clip extreme noise spikes: ○ Replace any value > 2.0 with 2.0. ○ Replace any value < -1.0 with -1.0.
- Return the cleaned NumPy array and calculate the standard deviation using np.std().

```
import numpy as np

signal = np.array([0.12, 0.85, 2.45, -1.20, 0.95, 3.10, -0.05, 0.60])
```

```
def clean_signal(sig_arr, lower_bound, upper_bound):
    sig_arr[sig_arr > upper_bound] = upper_bound
    sig_arr[sig_arr < lower_bound] = lower_bound
    return sig_arr

cleaned = clean_signal(signal, -1.0, 2.0)

print(cleaned)
print("Standard deviation:", np.std(cleaned))
```

```
[ 0.12  0.85  2.   -1.    0.95  2.   -0.05  0.6 ]
Standard deviation: 0.9527452112186133
```

Practical 29:

The Multi-Store Sales Comparison Engine

A franchise compares quarterly revenue (in lakhs) across 3 branch stores stored in a 2D NumPy array of shape (3, 4) (3 stores, 4 quarters): sales_data = np.array([[12, 15, 18, 20], [8, 10, 11, 14], [14, 12, 16, 19]]).

- Store branch names in a list: branches = ["Store North", "Store South", "Store East"].
- Calculate total annual revenue for each store by summing across rows (axis=1).
- Calculate total franchise revenue per quarter by summing across columns (axis=0).
- Identify which branch generated the highest overall annual revenue and print thebranch summary.

```
import numpy as np

sales_data = np.array([
    [12, 15, 18, 20],
    [8, 10, 11, 14],
    [14, 12, 16, 19]
])

branches = ["Store North", "Store South", "Store East"]

annual_revenue = np.sum(sales_data, axis=1)
quarter_revenue = np.sum(sales_data, axis=0)

highest = np.argmax(annual_revenue)

print("Annual revenue:", annual_revenue)
print("Quarterly revenue:", quarter_revenue)
print("Highest revenue:", branches[highest])
print("Revenue:", annual_revenue[highest])
```

```
Annual revenue: [65 43 61]
Quarterly revenue: [34 37 45 53]
Highest revenue: Store North
Revenue: 65
```

Practical 30:

The Smart Power Grid Outage Tracker

An electrical utility company tracks operational status codes for 8 regional grid zones over a 3-day weekend. Statuses are stored as tuples of zone names and 1D NumPy arrays of daily status codes (1 = Normal, 0 = Outage):

grid_data = [("Zone A", np.array([1, 1, 1])), ("Zone B", np.array([1, 0, 1])), ("Zone C", np.array([0, 0, 1]))].

- Iterate through grid_data, unpacking each zone name and status array.
- For each zone, calculate total operational days using np.sum().
- Store zone names in a dictionary categorizing them into "Stable" (3 operational days)or "Unstable" (fewer than 3 operational days).
- Print the final grid stability report for utility engineers.

```
import numpy as np

grid_data = [
    ("Zone A", np.array([1, 1, 1])),
    ("Zone B", np.array([1, 0, 1])),
    ("Zone C", np.array([0, 0, 1]))
]

report = {"Stable": [], "Unstable": []}

for zone, status in grid_data:
    operational_days = np.sum(status)

    if operational_days == 3:
        report["Stable"].append(zone)
    else:
        report["Unstable"].append(zone)

print("Grid Stability Report:")
print(report)
```

```
Grid Stability Report:
{'Stable': ['Zone A'], 'Unstable': ['Zone B', 'Zone C']}
```